C# Programming for Indian Students — Chapter 9: Strings

Nov 5, 2026 · @Vikash

9.1 What you will learn

Almost every program you write works with **text**: a student's name, a mobile number, an email address, a PIN code, a line read from a file. In C#, text is stored in a **string**. You have used strings since Chapter 1 with `Console.WriteLine`; in this chapter you learn to look inside them, search them, cut them apart, join them, compare them, format them and check that what a user typed is valid.

- Write string literals, escape sequences and verbatim strings.
- Read the length of a string and each character in it with `Length` and `[i]`.
- Change case and remove extra spaces with `ToUpper`, `ToLower` and `Trim`.
- Search a string with `Contains`, `StartsWith`, `EndsWith`, `IndexOf` and `LastIndexOf`.
- Cut and change a string with `Substring`, `Replace`, `Insert`, `Remove`, `PadLeft` and `PadRight`.
- Break a line into pieces with `Split` and join pieces with `string.Join`.
- Compare strings correctly, with and without case.
- Format numbers, money (₹) and dates in a neat table.
- Validate names, 10-digit mobile numbers and 6-digit PIN codes.
- Understand why strings never change, and when to use `StringBuilder`.

You have already met strings. In Chapter 3 you declared `string name = "Asha";`, in Chapter 4 you read input with `Console.ReadLine() ?? ""`, and in Chapter 8 you stored names in a `string[]`. A string is, in fact, very much like a read-only array of characters, so your array skills from Chapter 8 apply here directly.

9.2 Your first strings

A **string** is a sequence of characters written between double quotes. The text in quotes is a **string literal**; a variable of type `string` holds it.

```
string city = "Warangal";
string fullName = college + ", " + city;   // + joins (concatenates) strings
string batch = "Batch " + 2026;            // a number joined to a string
becomes text
```

Vikash Kumar

Some characters cannot be typed directly inside quotes, so C# uses **escape sequences** that start with a backslash.

Escape	Meaning	Example	Prints
`\n`	new line	`"One\nTwo"`	One, then Two on the next line
`\t`	tab	`"Name:\tRavi"`	Name: Ravi
`\"`	double quote	`"\"Namaste\""`	"Namaste"
`\\`	backslash	`"C:\\BCA"`	C:\BCA

A **verbatim string** starts with `@`. Inside it the backslash is an ordinary character, which is handy for Windows folder paths: `@"C:\Students\BCA"`. The **empty string** `""` has no characters; its length is 0.

Program 9.1 — First strings

```
string college = "National Institute of Technology";
string city = "Warangal";
string fullName = college + ", " + city;

Console.WriteLine(fullName);
Console.WriteLine($"Welcome to {city}!");

// Escape sequences
Console.WriteLine("Name:\tRavi Kumar");
Console.WriteLine("Line one\nLine two");
Console.WriteLine("He said \"Namaste\" to everyone.");
Console.WriteLine("Folder: C:\\Students\\BCA");

// Verbatim string: backslash is an ordinary character
Console.WriteLine(@"Folder: C:\Students\BCA");

// Empty string and joining a number
string empty = "";
int year = 2026;
string batch = "Batch " + year;
Console.WriteLine($"[{empty}] has length {empty.Length}");
Console.WriteLine(batch);

// Strings never change: a new string is made instead
string state = "kerala";
string upper = state.ToUpper();
Console.WriteLine($"state = {state}, upper = {upper}");
```

Vikash Kumar

Output

```
National Institute of Technology, Warangal
Welcome to Warangal!
Name:   Ravi Kumar
Line one
Line two
He said "Namaste" to everyone.
Folder: C:\Students\BCA
Folder: C:\Students\BCA
[] has length 0
Batch 2026
state = kerala, upper = KERALA
```

What to notice.

- `$"Welcome to {city}!"` is **string interpolation** (Chapter 3): the value inside `{ }` is placed into the text. It is easier to read than many `+` signs.
- The last two folder lines print the same text: `"C:\\Students"` needs doubled backslashes, `@"C:\Students"` does not.
- `state.ToUpper()` did **not** change `state`; it returned a new string `"KERALA"`. A string is **immutable**: once made, it never changes. Every string method gives back a new string, so you must store the result (section 9.11 explains why this matters).

9.3 Length and characters

A string is a row of `char` values numbered from **0**, exactly like an array. `Length` gives how many characters it has (spaces count too), and `text[i]` gives the character at index `i`. The last character is at `text.Length - 1`.

Index	0	1	2	3	4	5	6
`"Mumbai!"[i]`	M	u	m	b	a	i	!

You can **read** a character with `[i]`, but you cannot **change** one: `name[0] = 'R';` is a compile error (section 9.12). The `char` type has helper methods that test one character:

Method	Returns true when the character is	Example
`char.IsDigit(ch)`	0 to 9	`char.IsDigit('7')` → True
`char.IsLetter(ch)`	a letter	`char.IsLetter('k')` → True

Vikash Kumar

Method	Returns true when the character is	Example
`char.IsWhiteSpace(ch)`	a space, tab or new line	`char.IsWhiteSpace(' ')` → True
`char.IsUpper(ch)` / `char.IsLower(ch)`	a capital / small letter	`char.IsUpper('D')` → True
`char.ToUpper(ch)` / `char.ToLower(ch)`	(converts, returns a `char`)	`char.ToLower('A')` → a

Program 9.2 — Counting the characters in a sentence

```
string sentence = "Chandrayaan 3 landed in 2023";

Console.WriteLine($"Sentence : {sentence}");
Console.WriteLine($"Length   : {sentence.Length}");
Console.WriteLine($"First    : {sentence[0]}");
Console.WriteLine($"Last     : {sentence[sentence.Length - 1]}");

int vowels = 0, consonants = 0, digits = 0, spaces = 0;

foreach (char ch in sentence)
{
    if (char.IsDigit(ch))
    {
        digits++;
    }
    else if (char.IsWhiteSpace(ch))
    {
        spaces++;
    }
    else if (char.IsLetter(ch))
    {
        char lower = char.ToLower(ch);
        if (lower == 'a' || lower == 'e' || lower == 'i' || lower == 'o' ||
lower == 'u')
        {
            vowels++;
        }
        else
        {
            consonants++;
        }
    }
```

Vikash Kumar

```
}

Console.WriteLine($"Vowels {vowels}, consonants {consonants}, digits
{digits}, spaces {spaces}");

Console.Write("Reversed : ");
for (int i = sentence.Length - 1; i >= 0; i--)
{
    Console.Write(sentence[i]);
}
Console.WriteLine();
```

Output

```
Sentence : Chandrayaan 3 landed in 2023
Length   : 28
First    : C
Last     : 3
Vowels 7, consonants 12, digits 5, spaces 4
Reversed : 3202 ni dednal 3 naayardnahC
```

What to notice.

- `foreach (char ch in sentence)` visits every character in order, just as it visited every element of an array in Chapter 8.
- Characters use **single** quotes (`'a'`), strings use **double** quotes (`"a"`). `'a'` is a `char`; `"a"` is a `string` of length 1.
- `char.ToLower(ch)` lets one test cover both `'A'` and `'a'`. Every character falls into exactly one group, so the counts add up to `Length` (7 + 12 + 5 + 4 = 28).
- Reversing uses a `for` loop that counts **down** from `Length - 1` to `0`, the same pattern as printing an array backwards.

9.4 Changing case and trimming

Users type carelessly: extra spaces before and after a name, `YES`, `Yes` or `yes` for the same answer. Clean the input **before** you use it.

Method	What it returns	`" Rahul "` becomes
`Trim()`	spaces removed from both ends	`"Rahul"`
`TrimStart()`	spaces removed from the start only	`"Rahul "`
`TrimEnd()`	spaces removed from the end only	`" Rahul"`

Vikash Kumar

Method	What it returns	`" Rahul "` becomes
`ToUpper()`	every letter in capitals	`" RAHUL "`
`ToLower()`	every letter in small letters	`" rahul "`

`Trim` never removes spaces **inside** the text: `" rahul sharma ".Trim()` is `"rahul sharma"`.

Program 9.3 — Cleaning a name and an answer

```
Console.Write("Enter your name: ");
string typed = Console.ReadLine() ?? "";

string name = typed.Trim();

Console.WriteLine($"Typed   : [{typed}] ({typed.Length} characters)");
Console.WriteLine($"Trimmed : [{name}] ({name.Length} characters)");
Console.WriteLine($"Upper   : {name.ToUpper()}");
Console.WriteLine($"Lower   : {name.ToLower()}");
Console.WriteLine($"TrimStart: [{typed.TrimStart()}]");
Console.WriteLine($"TrimEnd  : [{typed.TrimEnd()}]");

Console.Write("Do you want a hostel room (yes/no)? ");
string answer = (Console.ReadLine() ?? "").Trim().ToLower();

if (answer == "yes")
{
    Console.WriteLine($"Hostel room booked for {name.ToUpper()}.");
}
else
{
    Console.WriteLine("No hostel room booked.");
}
```

Output (typed input: `rahul sharma` with 3 spaces before and 2 after, then `YES`)

```
Enter your name:    rahul sharma
Typed   : [   rahul sharma  ] (17 characters)
Trimmed : [rahul sharma] (12 characters)
Upper   : RAHUL SHARMA
Lower   : rahul sharma
TrimStart: [rahul sharma  ]
TrimEnd  : [   rahul sharma]
Do you want a hostel room (yes/no)?   YES
Hostel room booked for RAHUL SHARMA.
```

Vikash Kumar

What to notice.

- The square brackets in the output make the hidden spaces visible. Use this trick whenever you debug strings.
- `(Console.ReadLine() ?? "").Trim().ToLower()` is **method chaining**: each method works on the string returned by the one before it. After it, `" YES "`, `"Yes"` and `"yes"` all become `"yes"`.
- `typed` still holds the original text with spaces. Trimming created a new string in `name`.

9.5 Searching in a string

In Chapter 8 you wrote a linear search by hand. For strings, C# already has the searches built in. Three answer **yes or no** (`bool`), two answer **where** (`int`).

Method	Returns	`"priya.verma@gmail.com"`
`Contains(x)`	`true` if `x` appears anywhere	`Contains('@')` → True
`StartsWith(x)`	`true` if the string begins with `x`	`StartsWith("priya")` → True
`EndsWith(x)`	`true` if the string ends with `x`	`EndsWith(".com")` → True
`IndexOf(x)`	index of the **first** `x`, or **-1** if not found	`IndexOf('.')` → 5
`LastIndexOf(x)`	index of the **last** `x`, or **-1**	`LastIndexOf('.')` → 17

`x` can be one `char` (`'@'`) or a whole string (`"gmail"`). All five are **case-sensitive**: `"Hall"` and `"hall"` are different text.

Program 9.4 — Checking an email, vehicle numbers and a notice

```
string email = "priya.verma@gmail.com";

Console.WriteLine($"Email           : {email}");
Console.WriteLine($"Contains '@'    : {email.Contains('@')}");
Console.WriteLine($"Ends with '.com' : {email.EndsWith(".com")}");
Console.WriteLine($"Position of '@'  : {email.IndexOf('@')}");
Console.WriteLine($"Position of '.'  : {email.IndexOf('.')} (first),
{email.LastIndexOf('.')} (last)");
Console.WriteLine($"Position of 'x'  : {email.IndexOf('x')}");

string[] vehicles = { "KA01AB1234", "TN09CD4321", "KA05MN7777", "MH12XY0001"
};

Console.WriteLine();
```

Vikash Kumar

```
Console.WriteLine("Vehicles registered in Karnataka:");
foreach (string number in vehicles)
{
    if (number.StartsWith("KA"))
    {
        Console.WriteLine($"  {number}");
    }
}

string message = "Exam on Monday. Hall opens at 9 am.";
Console.WriteLine();
Console.WriteLine($"Contains \"hall\"      : {message.Contains("hall")}");
Console.WriteLine($"Contains \"Hall\"      : {message.Contains("Hall")}");
Console.WriteLine($"\"Monday\" starts at   : {message.IndexOf("Monday")}");
```

Output

```
Email            : priya.verma@gmail.com
Contains '@'     : True
Ends with '.com' : True
Position of '@'  : 11
Position of '.'  : 5 (first), 17 (last)
Position of 'x'  : -1

Vehicles registered in Karnataka:
  KA01AB1234
  KA05MN7777

Contains "hall"     : False
Contains "Hall"     : True
"Monday" starts at  : 8
```

What to notice.

- `IndexOf` returns **-1** when the character is missing, the same "not found" value you used for `position` in Chapter 8. Always check for -1 before using the result as an index.
- Indian vehicle numbers begin with the state code (KA, TN, MH), so `StartsWith("KA")` filters Karnataka vehicles in one line.
- `IndexOf("Monday")` gives the index of the **first character** of the word: `M` is at index 8.
- `Contains("hall")` is False because the notice says `Hall`. Section 9.8 shows how to search and compare without caring about case.

Vikash Kumar

9.6 Extracting and changing

`Substring` cuts a piece out of a string. It has two forms:

- `text.Substring(start, length)` — `length` characters beginning at index `start`.
- `text.Substring(start)` — everything from `start` to the end.

Index	0	1	2	3	4	5	6	7	8	9
`"BCA2026045"`	B	C	A	2	0	2	6	0	4	5
Piece	course	course	course	year	year	year	year	serial	serial	serial

So `Substring(0, 3)` is `"BCA"`, `Substring(3, 4)` is `"2026"` and `Substring(7)` is `"045"`. The second number is a **length**, not an end index; mixing the two up is the most common `Substring` mistake.

The other methods return a changed **copy**:

Method	What it does	Example → result
`Replace(old, new)`	replaces every `old` with `new`	`"Sat".Replace("Sat", "Mon")` → `"Mon"`
`Insert(index, text)`	puts `text` in before `index`	`"0511".Insert(2, "/")` → `"05/11"`
`Remove(start, count)`	deletes `count` characters from `start`	`"PIN-560001".Remove(0, 4)` → `"560001"`
`PadLeft(width, ch)`	adds `ch` on the left up to `width`	`"7".PadLeft(3, '0')` → `"007"`
`PadRight(width)`	adds spaces on the right up to `width`	`"Rice".PadRight(8)` → `"Rice "`

Program 9.5 — Roll numbers, email parts, masking and padding

```
string rollNumber = "BCA2026045";

string course = rollNumber.Substring(0, 3);
string year = rollNumber.Substring(3, 4);
string serial = rollNumber.Substring(7);

Console.WriteLine($"Roll number : {rollNumber}");
Console.WriteLine($"Course      : {course}");
Console.WriteLine($"Year        : {year}");
```

Vikash Kumar

```
Console.WriteLine($"Serial      : {serial}");

string email = "rahul.k@yahoo.in";
int at = email.IndexOf('@');
string userName = email.Substring(0, at);
string domain = email.Substring(at + 1);
Console.WriteLine($"User name   : {userName}, domain: {domain}");

string mobile = "9876543210";
string masked = mobile.Substring(0, 2) + "******" + mobile.Substring(8);
Console.WriteLine($"Masked      : {masked}");

string notice = "Classes will be held on Saturday.";
Console.WriteLine(notice.Replace("Saturday", "Monday"));

string date = "05112026";
string withSlashes = date.Insert(4, "/").Insert(2, "/");
Console.WriteLine($"Date        : {withSlashes}");

string code = "PIN-560001";
Console.WriteLine($"Without PIN-: {code.Remove(0, 4)}");

int seat = 7;
Console.WriteLine($"Seat number : {seat.ToString().PadLeft(3, '0')}");
Console.WriteLine($"[{"Rice".PadRight(8)}][{"60".PadLeft(5)}]");
```

Output

```
Roll number : BCA2026045
Course      : BCA
Year        : 2026
Serial      : 045
User name   : rahul.k, domain: yahoo.in
Masked      : 98******10
Classes will be held on Monday.
Date        : 05/11/2026
Without PIN-: 560001
Seat number : 007
[Rice    ][   60]
```

What to notice.

- `IndexOf` and `Substring` work as a team: find the position of `@`, then take everything before it and everything after it. The `@` itself (index `at`) is skipped by starting the domain at `at + 1`.

Vikash Kumar

- Masking shows only the first 2 and last 2 digits, the way banks and shopping sites print your mobile number on a receipt.
- `date.Insert(4, "/").Insert(2, "/")` inserts at the **later** position first, so the earlier index does not shift. Try the other order and see what you get.
- `PadLeft(3, '0')` turns seat 7 into `007`; padding is how fixed-width codes and neat columns are made.

9.7 Split and Join

Data often arrives as one line with a **separator** between the values, for example a line from a CSV file: `Ananya Iyer,BCA,82,76,91`. `Split` cuts the line at every separator and returns a `string[]`; `string.Join` does the opposite and glues an array back into one string.

Call	Result
`"Ananya Iyer,BCA,82,76,91".Split(',')`	`{ "Ananya Iyer", "BCA", "82", "76", "91" }` (5 pieces)
`"India is my country".Split(' ')`	`{ "India", "is", "my", "country" }`
`string.Join(", ", subjects)`	`"C#, DBMS, Maths"`

The pieces are still **strings**. To add marks, convert each piece with `int.Parse` (Chapter 4).

Program 9.6 — Reading a student record and reversing words

```
string record = "Ananya Iyer,BCA,82,76,91";

string[] parts = record.Split(',');

Console.WriteLine($"Pieces : {parts.Length}");
Console.WriteLine($"Name   : {parts[0]}");
Console.WriteLine($"Course : {parts[1]}");

int total = 0;
for (int i = 2; i < parts.Length; i++)
{
    total += int.Parse(parts[i]);
}
Console.WriteLine($"Total  : {total}");

string sentence = "India is my country";
```

Vikash Kumar

```
string[] words = sentence.Split(' ');
Console.WriteLine($"Words  : {words.Length}");
Console.WriteLine($"Last word: {words[words.Length - 1]}");

Array.Reverse(words);
Console.WriteLine($"Reversed order: {string.Join(" ", words)}");

string[] subjects = { "C#", "DBMS", "Maths" };
Console.WriteLine($"Subjects: {string.Join(", ", subjects)}");

string messy = "Pune,,Nagpur, ,Nashik";
string[] cities = messy.Split(',', StringSplitOptions.RemoveEmptyEntries |
StringSplitOptions.TrimEntries);
Console.WriteLine($"Cities: {string.Join(" | ", cities)} ({cities.Length})");
```

Output

```
Pieces : 5
Name   : Ananya Iyer
Course : BCA
Total  : 249
Words  : 4
Last word: country
Reversed order: country my is India
Subjects: C#, DBMS, Maths
Cities: Pune | Nagpur | Nashik (3)
```

What to notice.

- The loop starts at index 2 because pieces 0 and 1 are the name and course; only pieces 2 onwards are marks.
- `Array.Reverse` (Chapter 8) works on the words array, and `string.Join(" ", words)` turns it back into a sentence.
- Without options, `"Pune,,Nagpur, ,Nashik".Split(',')` gives **5** pieces, including an empty `""` and a single space `" "`. `RemoveEmptyEntries` drops empty pieces and `TrimEntries` trims each piece; together they leave the 3 real cities. The `|` joins the two options.

9.8 Comparing strings

In C#, `==` on two strings compares their **text**, character by character, so `"Delhi" == "Delhi"` is `true`. It is case-sensitive: `"Delhi" == "delhi"` is `false`. When case should not matter (a city name, a yes/no answer, an email), use `Equals` with `StringComparison.OrdinalIgnoreCase`.

Vikash Kumar

To put names in **dictionary order**, you need more than equal or not equal. `string.Compare(a, b)` returns a number:

`string.Compare(a, b)` returns	Meaning	Example
less than 0	`a` comes **before** `b`	`Compare("Amit", "Bina")` → -1
0	same text	`Compare("Amit", "Amit")` → 0
greater than 0	`a` comes **after** `b`	`Compare("Bina", "Amit")` → 1

`a.CompareTo(b)` gives the same answer. You cannot write `"Amit" < "Bina"`; the `<` and `>` operators do not work on strings (section 9.12).

Program 9.7 — Equal, ignore case, compare and sort

```
string a = "Delhi";
string b = "Delhi";
string c = "delhi";

Console.WriteLine($"a == b : {a == b}");
Console.WriteLine($"a == c : {a == c}");
Console.WriteLine($"a.Equals(c, OrdinalIgnoreCase) : {a.Equals(c,
StringComparison.OrdinalIgnoreCase)}");

Console.WriteLine();
Console.WriteLine($"Compare(\"Amit\", \"Bina\") : {string.Compare("Amit",
"Bina")}");
Console.WriteLine($"Compare(\"Bina\", \"Amit\") : {string.Compare("Bina",
"Amit")}");
Console.WriteLine($"Compare(\"Amit\", \"Amit\") : {string.Compare("Amit",
"Amit")}");

Console.WriteLine();
string[] names = { "Rohan", "aditi", "Kavya", "Arjun", "meera" };
Array.Sort(names, StringComparer.OrdinalIgnoreCase);
Console.WriteLine($"Sorted : {string.Join(", ", names)}");

Console.WriteLine();
Console.Write("Enter the password hint city: ");
string hint = (Console.ReadLine() ?? "").Trim();

if (hint.Equals("Jaipur", StringComparison.OrdinalIgnoreCase))
{
    Console.WriteLine("Correct hint.");
}
```

Vikash Kumar

```
else
{
    Console.WriteLine("Wrong hint.");
}
```

Output (typed input: `jaipur`)

```
a == b : True
a == c : False
a.Equals(c, OrdinalIgnoreCase) : True

Compare("Amit", "Bina") : -1
Compare("Bina", "Amit") : 1
Compare("Amit", "Amit") : 0

Sorted : aditi, Arjun, Kavya, meera, Rohan

Enter the password hint city: jaipur
Correct hint.
```

What to notice.

- In Java you must use `equals` because `==` compares references. In C#, `==` on strings already compares the text, so `a == b` is the normal way to check equality.
- **Ordinal** means "compare the character codes"; **IgnoreCase** treats `A` and `a` as the same. `StringComparison.OrdinalIgnoreCase` is the usual choice for codes, emails and answers.
- `Array.Sort(names, StringComparer.OrdinalIgnoreCase)` sorts without caring about case, so `aditi` and `meera` are placed among the capitalised names instead of being pushed to one end.
- Comparing with `ToLower()` on both sides also works (`hint.ToLower() == "jaipur"`), but it builds two new strings; `Equals` with `OrdinalIgnoreCase` does not.

9.9 Formatting strings

Inside an interpolated string, each `{ }` can carry two extras: an **alignment** after a comma and a **format** after a colon.

```
$"{value,width:format}"
```

- **Width** is the number of characters the value takes. A positive width right-aligns it (good for numbers), a negative width left-aligns it (good for names): `{"Item",-14}`, `{amount,12}`.

Vikash Kumar

- **Format** says how a number or date is written.

Format	Meaning	Value	Result (en-IN)
`N2`	number, grouping, 2 decimals	125000m	1,25,000.00
`N0`	number, grouping, no decimals	125000m	1,25,000
`C`	currency of the culture	125000m	₹1,25,000.00
`D3`	whole number, at least 3 digits	7	007
`P1`	percentage, 1 decimal	0.8675	86.8%
`dd-MM-yyyy`	day-month-year	5 Nov 2026	05-11-2026

The grouping depends on the **culture** (the regional settings) of the computer. India uses lakh grouping (1,25,000), the United States uses thousand grouping (125,000). Program 9.8 sets the culture to `en-IN` so that the output is the same on every computer.

Program 9.8 — A shop bill in rupees

```
using System.Globalization;
using System.Text;

Console.OutputEncoding = Encoding.UTF8;
CultureInfo.CurrentCulture = new CultureInfo("en-IN");

string[] items = { "Notebook", "Gel pen", "Calculator", "Geometry box" };
int[] quantity = { 6, 10, 1, 2 };
decimal[] price = { 45.50m, 10m, 1250m, 89.99m };

Console.WriteLine("BHARAT STATIONERY, BENGALURU");
Console.WriteLine($"{"Item",-14}{"Qty",5}{"Price",12}{"Amount",12}");
Console.WriteLine(new string('-', 43));

decimal total = 0;
for (int i = 0; i < items.Length; i++)
{
    decimal amount = quantity[i] * price[i];
    total += amount;
    Console.WriteLine($"{items[i],-14}{quantity[i],5}{price[i],12:N2}
{amount,12:N2}");
}

Console.WriteLine(new string('-', 43));
Console.WriteLine($"{"Total",-14}{total,29:N2}");
```

Vikash Kumar

```
Console.WriteLine();

decimal fee = 125000m;
Console.WriteLine($"Fee (N2)        : {fee:N2}");
Console.WriteLine($"Fee (C)         : {fee:C}");
Console.WriteLine($"Fee (N0)        : {fee:N0}");
Console.WriteLine($"Fee (invariant) : {fee.ToString("N2",
CultureInfo.InvariantCulture)}");

int rank = 7;
double percent = 0.8675;
Console.WriteLine($"Rank (D3)       : {rank:D3}");
Console.WriteLine($"Percent (P1)    : {percent:P1}");
Console.WriteLine($"Date            : {new DateTime(2026, 11, 5):dd-MM-
yyyy}");
```

Output

```
BHARAT STATIONERY, BENGALURU
Item            Qty       Price      Amount
-----------------------------------------
Notebook          6       45.50      273.00
Gel pen          10       10.00      100.00
Calculator        1    1,250.00    1,250.00
Geometry box      2       89.99      179.98
-----------------------------------------
Total                             1,802.98

Fee (N2)        : 1,25,000.00
Fee (C)         : ₹1,25,000.00
Fee (N0)        : 1,25,000
Fee (invariant) : 125,000.00
Rank (D3)       : 007
Percent (P1)    : 86.8%
Date            : 05-11-2026
```

What to notice.

- `new string('-', 43)` builds a string of 43 dashes; 43 is the sum of the column widths (14 + 5 + 12 + 12).
- The total line uses width 29 (5 + 12 + 12) so that `1,802.98` lines up under the Amount column.
- `Console.OutputEncoding = Encoding.UTF8;` lets the Windows console print the ₹ sign. Without it you may see `?` instead. If ₹ still shows as a box, change the console font in Visual Studio's terminal to Cascadia Mono or Consolas.

Vikash Kumar

- Money is stored in `decimal` (the `m` suffix), never in `double`, so 45.50 × 6 is exactly 273.00.
- `CultureInfo.InvariantCulture` is a fixed, region-free format. Use it when you write numbers to files that other programs will read.

9.10 Validating input

Every admission form, bank app and shopping site checks what you type before it accepts it. With `Length`, a loop over the characters and the `char` tests from section 9.3, you can write the same checks. Each rule becomes one method (Chapter 7) that returns `bool`.

Field	Rule used in this book	Valid	Invalid
Name	at least 2 characters after trimming, only letters and spaces	Sneha Patil	R2D2
Mobile number	exactly 10 digits, first digit 6, 7, 8 or 9	9876543210	5876543210
PIN code	exactly 6 digits, first digit not 0	560001	056001

Program 9.9 — Checking names, mobile numbers and PIN codes

```
string[] names = { "Sneha Patil", "R2D2", "  ", "Mohammed Irfan" };
string[] mobiles = { "9876543210", "98765 43210", "5876543210", "987654321"
};
string[] pinCodes = { "560001", "056001", "11001", "4000a1" };

Console.WriteLine("Names");
foreach (string n in names)
{
    Console.WriteLine($"  [{n}] -> {(IsValidName(n) ? "valid" :
"invalid")}");
}

Console.WriteLine("Mobile numbers");
foreach (string m in mobiles)
{
    Console.WriteLine($"  [{m}] -> {(IsValidMobile(m) ? "valid" :
"invalid")}");
}

Console.WriteLine("PIN codes");
foreach (string p in pinCodes)
```

Vikash Kumar

```
{
    Console.WriteLine($"  [{p}] -> {(IsValidPin(p) ? "valid" : "invalid")}");
}

// A name has at least 2 letters and only letters and spaces.
bool IsValidName(string text)
{
    string name = text.Trim();
    if (name.Length < 2)
    {
        return false;
    }
    foreach (char ch in name)
    {
        if (!char.IsLetter(ch) && ch != ' ')
        {
            return false;
        }
    }
    return true;
}

// An Indian mobile number has 10 digits and starts with 6, 7, 8 or 9.
bool IsValidMobile(string text)
{
    if (text.Length != 10 || !AllDigits(text))
    {
        return false;
    }
    return text[0] >= '6' && text[0] <= '9';
}

// A PIN code has 6 digits and does not start with 0.
bool IsValidPin(string text)
{
    return text.Length == 6 && AllDigits(text) && text[0] != '0';
}

bool AllDigits(string text)
{
    foreach (char ch in text)
    {
        if (!char.IsDigit(ch))
        {
            return false;
```

Vikash Kumar

```
        }
    }
    return true;
}
```

Output

```
Names
  [Sneha Patil] -> valid
  [R2D2] -> invalid
  [  ] -> invalid
  [Mohammed Irfan] -> valid
Mobile numbers
  [9876543210] -> valid
  [98765 43210] -> invalid
  [5876543210] -> invalid
  [987654321] -> invalid
PIN codes
  [560001] -> valid
  [056001] -> invalid
  [11001] -> invalid
  [4000a1] -> invalid
```

What to notice.

- Each method **returns false as soon as one rule fails**; only a string that passes every test reaches `return true`. This is the "assume valid, look for a reason to reject" pattern.
- `AllDigits` is written once and used by two other methods, the reuse idea from Chapter 7.
- `text[0] >= '6' && text[0] <= '9'` compares **characters**. Digit characters are stored in order ('0' < '1' < ... < '9'), so a range test works.
- The `&&` in `IsValidPin` stops at the first false part (short-circuit, Chapter 5). If the length is wrong, `text[0]` is never read, so an empty string cannot crash it.
- These rules are simplified for learning. A real form also handles a leading `+91` or `0`, and names with a dot (`A. R. Rahman`); you will add such cases in the practice questions.

9.11 Strings are immutable: StringBuilder

A string **never changes** after it is made. `line += " there";` looks as if it adds to `line`, but C# actually builds a **new** string `"Hi there"` and points `line` at it; the old `"Hi"` is left for the garbage collector. That is fine a few times, but a loop that does `result += ...` a

Vikash Kumar

thousand times builds a thousand strings, each a little longer than the one before.

`StringBuilder` (in `System.Text`) keeps **one** growing buffer of characters that you can change in place, then turns it into a string once, with `ToString()`.

`StringBuilder` member	What it does
`Append(x)`	adds `x` (text, number, char) at the end
`AppendLine()`	adds a new line at the end
`Insert(index, x)`	puts `x` in at `index`
`Replace(old, new)`	replaces text inside the buffer
`Length`	number of characters so far
`ToString()`	gives the finished `string`

Program 9.10 — += versus StringBuilder

```
using System.Text;

// Every += makes a brand new string
string line = "Hi";
string before = line;
line += " there";
Console.WriteLine($"before = {before}, line = {line}");
Console.WriteLine($"Same object? {ReferenceEquals(before, line)}");

// StringBuilder changes one buffer in place
StringBuilder table = new StringBuilder();
for (int i = 1; i <= 5; i++)
{
    table.Append($"7 x {i} = {7 * i}");
    table.AppendLine();
}
Console.Write(table.ToString());

StringBuilder sb = new StringBuilder("Kolkata");
sb.Insert(0, "City: ");
sb.Append(", West Bengal");
sb.Replace("City", "Place");
Console.WriteLine(sb.ToString());

// Build a string of 1 to 10 separated by commas
StringBuilder numbers = new StringBuilder();
```

Vikash Kumar

```
for (int n = 1; n <= 10; n++)
{
    if (numbers.Length > 0)
    {
        numbers.Append(", ");
    }
    numbers.Append(n);
}
Console.WriteLine(numbers);
Console.WriteLine($"Length: {numbers.Length}");
```

Output

```
before = Hi, line = Hi there
Same object? False
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
Place: Kolkata, West Bengal
1, 2, 3, 4, 5, 6, 7, 8, 9, 10
Length: 29
```

What to notice.

- `ReferenceEquals(before, line)` is False: after `+=`, `line` points to a different string object, and `before` still holds the untouched `"Hi"`.
- `if (numbers.Length > 0)` adds the comma **before** every number except the first, so there is no extra comma at the end. (For an array, `string.Join` does the same job in one line.)
- `Console.WriteLine(numbers)` calls `ToString()` for you.

Use	When
`string` with `+` or `$"..."`	a few joins, text that is built once
`StringBuilder`	text built up inside a loop, many pieces, reports

9.12 Dry run and common errors

Dry run. A **palindrome** reads the same forwards and backwards (RADAR, MALAYALAM, 12321). Program 9.11 checks one with two indexes: `left` starts at the first character, `right` at the last, and they walk towards each other.

Vikash Kumar

Program 9.11 — Palindrome check

```
string word = "RADAR";
bool isPalindrome = true;
int left = 0;
int right = word.Length - 1;

while (left < right)
{
    if (word[left] != word[right])
    {
        isPalindrome = false;
        break;
    }
    left++;
    right--;
}

Console.WriteLine($"{word} palindrome? {isPalindrome}");
```

Step	left	right	`left < right`	word[left]	word[right]	Same?	isPalindrome
start	0	4					true
1	0	4	true	R	R	yes	true
2	1	3	true	A	A	yes	true
3	2	2	false	(loop ends)			true

Output

```
RADAR palindrome? True
```

The middle `D` is never compared with anything, and it does not need to be. With `"RAJAN"`, step 1 compares `R` with `N`, they differ, `isPalindrome` becomes false and `break` stops the loop at once.

Common compile-time errors. The messages below are real .NET 10 build output.

Wrong code	Error message	Fix
`name[0] = 'R';`	CS0200: Property or indexer 'string.this[int]' cannot be assigned to -- it is read only	Strings are immutable. Build a new string: `name = "R" + name.Substring(1);`

Vikash Kumar

Wrong code	Error message	Fix
`char grade = "A";`	CS0029: Cannot implicitly convert type 'string' to 'char'	A `char` uses single quotes: `'A'`.
`string city = 'Delhi';`	CS1012: Too many characters in character literal	A string uses double quotes: `"Delhi"`.
`s.length`	CS1061: 'string' does not contain a definition for 'length' ...	C# is case-sensitive: `s.Length`.
`"Amit" < "Bina"`	CS0019: Operator '<' cannot be applied to operands of type 'string' and 'string'	Use `string.Compare("Amit", "Bina") < 0`.
`s - "na"`	CS0019: Operator '-' cannot be applied to operands of type 'string' and 'string'	Only `+` works on strings. Use `Replace` or `Substring`.
`string total = 250;`	CS0029: Cannot implicitly convert type 'int' to 'string'	`string total = 250.ToString();` or `$"{250}"`.
`string msg = "Hello;`	CS1010: Newline in constant (plus CS1002: ; expected)	Close the string: `"Hello";`
`string? city = null;` then `city.Length`	warning CS8602: Dereference of a possibly null reference.	Check first: `if (city != null)`, or use `city ?? ""`.

Run-time errors. These compile, then stop the program when it runs.

1. `"BCA2026045".Substring(7, 5)` → **System.ArgumentOutOfRangeException**: Index and length must refer to a location within the string. Only 3 characters remain after index 7. Check `start + length <= text.Length` first.
2. `"Agra"[4]` → **System.IndexOutOfRangeException**: Index was outside the bounds of the array. The last index of a 4-letter word is 3, exactly as with arrays.
3. `int.Parse("8a")` → **System.FormatException**: The input string '8a' was not in a correct format. Validate with `AllDigits` (Program 9.9) or use `int.TryParse`.
4. Using `.Length` on a string that is really `null` → **System.NullReferenceException**: Object reference not set to an instance of an object. This is why the book reads input with `Console.ReadLine() ?? ""`, which never gives null.

Vikash Kumar

9.13 Mini project: College Registration Desk

The registration desk of a college types each new student's details. The program refuses bad input politely, keeps asking until each field is right, and then prints a neat registration card with a generated roll number. It uses almost every string skill in this chapter.

Requirements

1. Read the full name, mobile number, PIN code, email, course (BCA, BSC or BTECH, any case) and a serial number from 1 to 999.
2. For every field, trim the input, check it, and on bad input print a hint and ask again.
3. Build the roll number as `COURSE-2026-SSS`, for example `BTECH-2026-007`.
4. Print the name in Title Case, the initials, the masked mobile number, and the email in small letters, inside a bordered card.

String skills used

Skill	Where it is used
`Trim`, `ToUpper`, `ToLower`	cleaning every input; roll number; email
`Length`, `[i]`, loop over `char`	`IsValidName`, `IsValidMobile`, `IsValidPin`, `AllDigits`
`IndexOf`, `LastIndexOf`, `Contains`	`IsValidEmail`
`Equals(..., OrdinalIgnoreCase)`	`IsValidCourse`
`Split`, `Substring`, `string.Join`	`ToTitleCase`, `Mask`
`PadLeft`, alignment `{x,-12}`	serial `007`, card layout
`StringBuilder`	building the card and the initials

Methods used

Method	Job
`ReadValid(prompt, field, hint)`	asks again and again until the text is valid, then returns it
`IsValid(field, text)`	picks the right check for the field with a `switch`
`IsValidName` / `IsValidMobile` / `IsValidPin`	the rules from section 9.10

Vikash Kumar

Method	Job
`IsValidEmail`	one `@` with text before it, a `.` after it, no spaces
`IsValidCourse` / `IsValidSerial`	BCA, BSC or BTECH; 1 to 3 digits, at least 1
`AllDigits`	every character is 0 to 9
`ToTitleCase` / `Initials` / `Mask`	formatting for the card

Program 9.12 — College Registration Desk

```
using System.Text;

Console.WriteLine("=== College Registration Desk ===");

string name = ReadValid("Full name      : ", "name", "Use letters and spaces
only, at least 2 letters.");
string mobile = ReadValid("Mobile number  : ", "mobile", "Enter 10 digits
starting with 6, 7, 8 or 9.");
string pin = ReadValid("PIN code     : ", "pin", "Enter 6 digits; the first
digit cannot be 0.");
string email = ReadValid("Email        : ", "email", "Enter an email like
name@college.ac.in.");
string course = ReadValid("Course (BCA/BSC/BTECH): ", "course", "Choose BCA,
BSC or BTECH.");
string serial = ReadValid("Serial number (1-999): ", "serial", "Enter a
number from 1 to 999.");

string rollNumber = $"{course.ToUpper()}-2026-{serial.PadLeft(3, '0')}";

StringBuilder card = new StringBuilder();
string border = new string('=', 40);
card.AppendLine(border);
card.AppendLine("REGISTRATION CARD".PadLeft(28));
card.AppendLine(border);
card.AppendLine($"{"Roll number",-12}: {rollNumber}");
card.AppendLine($"{"Name",-12}: {ToTitleCase(name)}");
card.AppendLine($"{"Initials",-12}: {Initials(name)}");
card.AppendLine($"{"Mobile",-12}: {Mask(mobile)}");
card.AppendLine($"{"PIN code",-12}: {pin}");
card.AppendLine($"{"Email",-12}: {email.ToLower()}");
card.Append(border);

Console.WriteLine();
Console.WriteLine(card);
```

Vikash Kumar

```
// Keeps asking until the typed text passes the check.
string ReadValid(string prompt, string field, string hint)
{
    while (true)
    {
        Console.Write(prompt);
        string text = (Console.ReadLine() ?? "").Trim();
        if (IsValid(field, text))
        {
            return text;
        }
        Console.WriteLine($"  Invalid. {hint}");
    }
}

// Picks the right check for each field.
bool IsValid(string field, string text)
{
    switch (field)
    {
        case "name": return IsValidName(text);
        case "mobile": return IsValidMobile(text);
        case "pin": return IsValidPin(text);
        case "email": return IsValidEmail(text);
        case "course": return IsValidCourse(text);
        case "serial": return IsValidSerial(text);
        default: return false;
    }
}

bool IsValidName(string text)
{
    if (text.Length < 2)
    {
        return false;
    }
    foreach (char ch in text)
    {
        if (!char.IsLetter(ch) && ch != ' ')
        {
            return false;
        }
    }
    return true;
}
```

Vikash Kumar

```
bool IsValidMobile(string text)
{
    return text.Length == 10 && AllDigits(text) && text[0] >= '6' && text[0]
<= '9';
}

bool IsValidPin(string text)
{
    return text.Length == 6 && AllDigits(text) && text[0] != '0';
}

// One '@', something before it, and a '.' somewhere after it, no spaces.
bool IsValidEmail(string text)
{
    int at = text.IndexOf('@');
    if (at <= 0 || at != text.LastIndexOf('@') || text.Contains(' '))
    {
        return false;
    }
    int dot = text.LastIndexOf('.');
    return dot > at + 1 && dot < text.Length - 1;
}

bool IsValidCourse(string text)
{
    return text.Equals("BCA", StringComparison.OrdinalIgnoreCase)
        || text.Equals("BSC", StringComparison.OrdinalIgnoreCase)
        || text.Equals("BTECH", StringComparison.OrdinalIgnoreCase);
}

bool IsValidSerial(string text)
{
    return text.Length >= 1 && text.Length <= 3 && AllDigits(text) &&
int.Parse(text) >= 1;
}

// Only the digits 0 to 9 (char.IsDigit also accepts digits of other
scripts).
bool AllDigits(string text)
{
    foreach (char ch in text)
    {
        if (ch < '0' || ch > '9')
        {
            return false;
```

Vikash Kumar

```
        }
    }
    return true;
}

// "aMAN   kumar singh" -> "Aman Kumar Singh"
string ToTitleCase(string text)
{
    string[] words = text.Split(' ', StringSplitOptions.RemoveEmptyEntries);
    for (int i = 0; i < words.Length; i++)
    {
        string w = words[i].ToLower();
        words[i] = char.ToUpper(w[0]) + w.Substring(1);
    }
    return string.Join(" ", words);
}

// "aman kumar singh" -> "A. K. S."
string Initials(string text)
{
    StringBuilder result = new StringBuilder();
    foreach (string w in text.Split(' ',
StringSplitOptions.RemoveEmptyEntries))
    {
        result.Append(char.ToUpper(w[0])).Append(". ");
    }
    return result.ToString().TrimEnd();
}

string Mask(string number)
{
    return number.Substring(0, 2) + new string('*', number.Length - 4) +
number.Substring(number.Length - 2);
}
```

Output (typed input, one per prompt: `aMAN kumar singh`; `98765-43210`, then `9876543210`; `056001`, then `560034`; `aman.singh@gmail`, then `Aman.Singh@Gmail.com`; `mca`, then `btech`; `7`)

```
=== College Registration Desk ===
Full name      : aMAN   kumar singh
Mobile number  : 98765-43210
  Invalid. Enter 10 digits starting with 6, 7, 8 or 9.
Mobile number  : 9876543210
PIN code       : 056001
  Invalid. Enter 6 digits; the first digit cannot be 0.
```

Vikash Kumar

```
PIN code       : 560034
Email          : aman.singh@gmail
  Invalid. Enter an email like name@college.ac.in.
Email          : Aman.Singh@Gmail.com
Course (BCA/BSC/BTECH): mca
  Invalid. Choose BCA, BSC or BTECH.
Course (BCA/BSC/BTECH): btech
Serial number (1-999): 7

========================================
           REGISTRATION CARD
========================================
Roll number : BTECH-2026-007
Name        : Aman Kumar Singh
Initials    : A. K. S.
Mobile      : 98******10
PIN code    : 560034
Email       : aman.singh@gmail.com
========================================
```

Design decisions.

- `ReadValid` holds the only input loop. Adding a new field means writing one new `IsValid...` method and one `case`, not another `while` loop.
- `while (true)` with `return` inside is safe here: the loop ends only when the text is valid, which is exactly what the caller needs.
- `AllDigits` checks `'0'` to `'9'` directly. `char.IsDigit` (used in Program 9.9) also returns true for digits of other scripts such as Devanagari `५`, which `int.Parse` would then reject with a `FormatException`.
- `aman.singh@gmail` is rejected because there is no `.` after the `@`. `IsValidEmail` is a beginner's check; real systems confirm an email by sending a message to it.
- Split with `RemoveEmptyEntries` lets `ToTitleCase` cope with the three spaces between `aMAN` and `kumar`.
- The card is built in a `StringBuilder` and printed once, so it could just as easily be saved to a file in a later chapter.

9.14 Practice questions with answers

Try every question yourself before you read the answer. Programs for Q6 to Q14 are in the `Solutions` folder of this chapter on GitHub.

Vikash Kumar

Q1. What does "strings are immutable" mean? Once a string is created, its characters can never be changed. Methods such as `ToUpper`, `Replace` and `Trim` return a **new** string; the original stays as it was. That is why you must store the result: `name = name.Trim();`.

Q2. What is the difference between `'A'` and `"A"`? `'A'` is a `char`, exactly one character, in single quotes. `"A"` is a `string` of length 1, in double quotes. `char grade = "A";` does not compile (CS0029).

Q3. What does `IndexOf` return when the text is not found, and why does it matter? It returns **-1**. If you pass -1 to `Substring` or use it as an index, the program crashes with `ArgumentOutOfRangeException` or `IndexOutOfRangeException`, so check for -1 first.

Q4. `"Hyderabad".Substring(2, 3)` gives what? And `"Hyderabad".Substring(5)`? `"der"` (3 characters from index 2) and `"abad"` (from index 5 to the end).

Q5. When should you use `StringBuilder` instead of `+`? When a string is built up inside a loop or from many pieces, such as a report or a table. Each `+` creates a new string; `StringBuilder` changes one buffer and makes the string once with `ToString()`.

Q6. Predict the output.

```
string s = "Programming";
Console.WriteLine(s.Length);
Console.WriteLine(s[3]);
Console.WriteLine(s.Substring(3, 4));
Console.WriteLine(s.IndexOf('g') + " " + s.LastIndexOf('g'));
Console.WriteLine(s.ToUpper().Replace("G", "g"));
```

```
11
g
gram
3 10
PROgRAMMINg
```

Indexes: P0 r1 o2 g3 r4 a5 m6 m7 i8 n9 g10. `Substring(3, 4)` takes 4 characters starting at the first `g`. `ToUpper` gives `PROGRAMMING`, then `Replace` turns both capital `G`s into `g`.

Q7. Predict the output.

Vikash Kumar

```
string a = "Hello";
string b = a;
b += " World";
Console.WriteLine(a);
Console.WriteLine(b);
Console.WriteLine(a.Length + b.Length);
Console.WriteLine("5" + 5 + 5);
Console.WriteLine(5 + 5 + "5");
```

```
Hello
Hello World
16
555
105
```

`b += " World"` makes a new string for `b`; `a` is unchanged (compare arrays in Chapter 8, where changing `q` also changed `p`). `+` works left to right: `"5" + 5` is already text `"55"`, so the next 5 is joined; in `5 + 5 + "5"`, the numbers are added first (10), then joined.

Q8. Predict the output.

```
string[] p = "10-20-30".Split('-');
Console.WriteLine(p[0] + p[1]);
Console.WriteLine(int.Parse(p[0]) + int.Parse(p[1]));
Console.WriteLine(string.Join("+", p));
Console.WriteLine("abc".CompareTo("abd"));
Console.WriteLine("  Hi  ".Trim().Length);
```

```
1020
30
10+20+30
-1
2
```

The pieces are strings, so `p[0] + p[1]` joins them; after `int.Parse` they add. `"abc"` comes before `"abd"`, so `CompareTo` is negative.

Q9. Write a program that checks whether a sentence is a palindrome, ignoring spaces, punctuation and case.

```
Console.Write("Enter a sentence: ");
string sentence = Console.ReadLine() ?? "";

string letters = "";
foreach (char ch in sentence)
```

Vikash Kumar

```
{
    if (char.IsLetterOrDigit(ch))
    {
        letters += char.ToLower(ch);
    }
}

bool isPalindrome = true;
for (int i = 0; i < letters.Length / 2; i++)
{
    if (letters[i] != letters[letters.Length - 1 - i])
    {
        isPalindrome = false;
        break;
    }
}

Console.WriteLine($"Letters only: {letters}");
Console.WriteLine(isPalindrome ? "It is a palindrome." : "It is not a
palindrome.");
```

```
Enter a sentence: Never odd or even
Letters only: neveroddoreven
It is a palindrome.
```

First keep only letters and digits in small letters, then compare character `i` with character `Length - 1 - i`. Only half the string needs checking. (`+=` in a loop is acceptable here because a sentence is short; for long text use `StringBuilder`.)

Q10. Count the words in a sentence and find the longest word. Several spaces may separate words.

Vikash Kumar

```
Console.Write("Enter a sentence: ");
string sentence = Console.ReadLine() ?? "";

string[] words = sentence.Split(' ', StringSplitOptions.RemoveEmptyEntries);

Console.WriteLine($"Words: {words.Length}");

string longest = "";
foreach (string word in words)
{
    if (word.Length > longest.Length)
    {
        longest = word;
    }
}
Console.WriteLine($"Longest word: {longest} ({longest.Length} letters)");
```

```
Enter a sentence:   Bharat   is a land of  festivals
Words: 6
Longest word: festivals (9 letters)
```

`RemoveEmptyEntries` is the key: without it, the extra spaces would produce empty pieces and the count would be wrong.

Q11. Reverse every word of a sentence but keep the words in their places ("Hello India" → "olleH aidnI").

```
Console.Write("Enter a sentence: ");
string sentence = Console.ReadLine() ?? "";

string[] words = sentence.Split(' ', StringSplitOptions.RemoveEmptyEntries);

for (int i = 0; i < words.Length; i++)
{
    char[] letters = words[i].ToCharArray();
    Array.Reverse(letters);
    words[i] = new string(letters);
}

Console.WriteLine(string.Join(" ", words));
```

```
Enter a sentence: Hello India 2026
olleH aidnI 6202
```

Vikash Kumar

`ToCharArray()` copies a string into a `char[]` that **can** be changed; `new string(letters)` turns the array back into a string. Compare Program 9.6, which reversed the **order** of the words instead.

Q12. Count how many times each letter appears in a word, using an array of 26 counters.

```
Console.Write("Enter a word or sentence: ");
string text = (Console.ReadLine() ?? "").ToLower();

int[] count = new int[26];
foreach (char ch in text)
{
    if (ch >= 'a' && ch <= 'z')
    {
        count[ch - 'a']++;
    }
}

for (int i = 0; i < count.Length; i++)
{
    if (count[i] > 0)
    {
        Console.WriteLine($"{(char)('a' + i)} : {count[i]}");
    }
}
```

```
Enter a word or sentence: Bengaluru
a : 1
b : 1
e : 1
g : 1
l : 1
n : 1
r : 1
u : 2
```

`ch - 'a'` turns a letter into a number: `'a'` → 0, `'b'` → 1, … `'z'` → 25, which is exactly an index into `count` (the rating counter of Chapter 8, Q11). `(char)('a' + i)` goes back from the index to the letter.

Q13. People type mobile numbers as `+91 98765 43210`, `09876543210` or `+91-7012345678`. Write a method that returns the plain 10-digit number, or `""` if the number is invalid.

Vikash Kumar

```
string[] typed = { "9876543210", "+91 98765 43210", "09876543210", "+91-
7012345678", "12345" };

foreach (string number in typed)
{
    string result = Normalise(number);
    Console.WriteLine($"{number,-16} -> {(result == "" ? "invalid" :
result)}");
}

// Returns the plain 10-digit number, or "" if it is not a valid Indian
mobile number.
string Normalise(string text)
{
    string digits = text.Replace(" ", "").Replace("-", "");

    if (digits.StartsWith("+91"))
    {
        digits = digits.Substring(3);
    }
    else if (digits.Length == 11 && digits.StartsWith("0"))
    {
        digits = digits.Substring(1);
    }

    if (digits.Length != 10 || digits[0] < '6' || digits[0] > '9')
    {
        return "";
    }
    foreach (char ch in digits)
    {
        if (ch < '0' || ch > '9')
        {
            return "";
        }
    }
    return digits;
}
```

```
9876543210       -> 9876543210
+91 98765 43210  -> 9876543210
09876543210      -> 9876543210
+91-7012345678   -> 7012345678
12345            -> invalid
```

Vikash Kumar

Clean first (remove spaces and dashes), then remove a known prefix, then validate what is left. Doing the steps in this order keeps each step simple.

Q14. Two words are anagrams if they use the same letters (Listen and Silent). Write a program to check two words.

```
Console.Write("First word : ");
string first = Console.ReadLine() ?? "";
Console.Write("Second word: ");
string second = Console.ReadLine() ?? "";

string a = SortedLetters(first);
string b = SortedLetters(second);

Console.WriteLine($"Sorted letters: {a} and {b}");
Console.WriteLine(a == b ? "They are anagrams." : "They are not anagrams.");

string SortedLetters(string word)
{
    char[] letters = word.Replace(" ", "").ToLower().ToCharArray();
    Array.Sort(letters);
    return new string(letters);
}
```

```
First word : Listen
Second word: Silent
Sorted letters: eilnst and eilnst
They are anagrams.
```

If two words have the same letters, sorting the letters of each gives the same string, so one `==` answers the question. Try `Dormitory` and `Dirty room`.

Vikash Kumar

9.15 MCQs and viva questions

Multiple-choice questions

1. What is `"Chennai".Length`? (a) 6 (b) 7 (c) 8 (d) 0
2. What is the index of the last character of a string `s`? (a) `s.Length` (b) `s.Length + 1` (c) `s.Length - 1` (d) `0`
3. What does `"Kanpur".IndexOf('z')` return? (a) 0 (b) null (c) -1 (d) an exception
4. What is `"Bhopal".Substring(1, 3)`? (a) "Bho" (b) "hop" (c) "hopa" (d) "opa"
5. Which line does **not** compile? (a) `string s = "A";` (b) `char c = 'A';` (c) `char c = "A";` (d) `string s = "";`

6. After `string t = "goa"; t.ToUpper();` what is `t`? (a) "GOA" (b) "goa" (c) "Goa" (d) an empty string
7. What does `"a,b,,c".Split(',').Length` give? (a) 3 (b) 4 (c) 5 (d) 2
8. Which call compares `"KOCHI"` and `"kochi"` and returns `true`? (a) `"KOCHI" == "kochi"` (b) `"KOCHI".Equals("kochi")` (c) `"KOCHI".Equals("kochi", StringComparison.OrdinalIgnoreCase)` (d) `string.Compare("KOCHI", "kochi") > 0`
9. With the culture set to en-IN, what does `$"{250000m:N0}"` print? (a) 250,000 (b) 2,50,000 (c) 250000.00 (d) ₹2,50,000
10. Which namespace holds `StringBuilder`? (a) System.IO (b) System.Linq (c) System.Text (d) System.Collections

Answers: 1 (b), 2 (c), 3 (c), 4 (b), 5 (c), 6 (b) — the new string from `ToUpper` was not stored, 7 (b) — the empty piece between the two commas counts, 8 (c), 9 (b), 10 (c).

Viva questions

1. **Is `string` a value type or a reference type?** A reference type. But because strings are immutable and `==` compares the text, they behave in everyday code like values.
2. **Why can't you write `name[0] = 'R';`?** The indexer of `string` is read-only (CS0200), because strings are immutable. Build a new string instead.
3. **What is the difference between `Substring(2, 3)` and `Substring(2)`?** The first takes 3 characters from index 2; the second takes everything from index 2 to the end.
4. **What is an escape sequence? Give two examples.** A backslash code for a character that cannot be typed directly inside quotes: `\n` for a new line, `\t` for a tab, `\"` for a double quote.
5. **What is a verbatim string?** A string that starts with `@`, in which backslashes are ordinary characters: `@"C:\Students"`.
6. **How do you check whether a typed answer is "yes" whatever its case and spaces?** `answer.Trim().Equals("yes", StringComparison.OrdinalIgnoreCase)`.
7. **Why does the book read input with `Console.ReadLine() ?? ""`?** `ReadLine` can return `null` when the input ends. `?? ""` replaces `null` with an empty string, so calling `Trim` or `Length` on the result never throws `NullReferenceException`.

9.16 Summary

- A **string** is an immutable sequence of `char` values indexed from 0. `Length` gives its size; `s[i]` reads a character but cannot change it.
- Escape sequences (`\n`, `\t`, `\"`, `\\`) and verbatim strings (`@"..."`) write special characters.

Vikash Kumar

- `Trim`, `ToUpper` and `ToLower` clean input; always store the returned string.
- `Contains`, `StartsWith`, `EndsWith` answer yes or no; `IndexOf` and `LastIndexOf` give a position or **-1**.
- `Substring(start, length)` cuts; `Replace`, `Insert`, `Remove`, `PadLeft` and `PadRight` return changed copies.
- `Split` breaks a line into a `string[]`; `string.Join` glues an array into one string.
- `==` compares text with case; `Equals(..., StringComparison.OrdinalIgnoreCase)` ignores case; `string.Compare` gives dictionary order.
- `{value,width:format}` aligns and formats; the culture decides grouping and the currency sign (en-IN gives ₹1,25,000.00).
- Validation methods loop over characters and return `false` at the first broken rule.
- Use `StringBuilder` when text is built in a loop or from many pieces.

Next: Chapter 10, Object-Oriented Programming. So far every program has kept a student's name, marks and mobile number in separate variables or parallel arrays. In Chapter 10 you will create your own `Student` class that keeps all of them together, with methods such as `IsValidMobile` living inside the class itself.

Vikash Kumar